Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Problem statement 1: Create a list and perform the following methods 1) insert() 2) remove() 3) append() 4) len() 5) pop() 6) clear().

Explanation: To understand Python lists and learn how to insert, remove, add, count, and delete elements using built-in list methods.

Code:

```
 numbers = [10, 20, 30, 40]
print("Original List:", numbers)
numbers.insert(1, 15)
print("After insert:", numbers)
numbers.remove(30)
print("After remove:", numbers)
numbers.append(50)
print("After append:", numbers)
print("Length of List:", len(numbers))
numbers.pop()
print("After pop:", numbers)
numbers.clear()
print("After clear:", numbers)
```

Output:

```
Original List: [10, 20, 30, 40]
After insert: [10, 15, 20, 30, 40]
After remove: [10, 15, 20, 40]
After append: [10, 15, 20, 40, 50]
Length of List: 5
After pop: [10, 15, 20, 40]
After clear: []
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Problem statement 2: Create a dictionary and apply the following methods 1) Print the dictionary items 2) access items 3) use get() 4)change values 5) use len().

Explanation: To understand Python dictionaries and learn how to store data as key-value pairs, access values, update information, and count dictionary entries.

Code:

```
student = {
   "name": "Rahul",
   "age": 20,
   "marks": 85 }
print("Dictionary Items:", student.items())
print("Student Name:", student["name"])
print("Updated Dictionary:", student)
print("Length of Dictionary:", len(student))
```

Output:

```
Dictionary Items: dict_items([('name', 'Rahul'), ('age', 20), ('marks', 85)])
Student Name: Rahul
Updated Dictionary: {'name': 'Rahul', 'age': 20, 'marks': 85}
Length of Dictionary: 3
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Problem statement 3: Define two matrices. Find their sum, difference, transpose and product of two matrices.

Explanation: To perform basic mathematical operations on matrices using the NumPy library in Python.

Code:

```
import numpy as np
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])
print("Matrix A:")
print(A)
print("Matrix B:")
print(B)
print("Sum of Matrices:")
print(A + B)
print("Difference of Matrices:")
print(A - B)
print("Transpose of Matrix A:")
print(A.T)
print("Transpose of Matrix B:")
print(B.T)
print("Product of Matrices:")
print(np.dot(A, B))
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Output:

```
Matrix A:
[[1 2]
 [3 4]]
Matrix B:
[[5 6]
 [7 8]]
Sum of Matrices:
[[ 6  8]
 [10 12]]
Difference of Matrices:
[[-4 -4]
 [-4 -4]]
Transpose of Matrix A:
[[1 3]
 [2 4]]
Transpose of Matrix B:
[[5 7]
 [6 8]]
Product of Matrices:
[[19 22]
 [43 50]]
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Problem statement 4: Create a dataframe using dictionary contains student name and marks, and display students scoring more than 80 marks.

Explanation: To learn how to create a Pandas DataFrame and filter records based on a specified condition.

Code:

```
import pandas as pd
data = {
   "Name": ["Rahul", "Priya", "Aman", "Neha", "Rohit"],
   "Marks": [75, 90, 65, 85, 95]
}
df = pd.DataFrame(data)


print("Student Data:")
print(df)
print("\nStudents Scoring More Than 80 Marks:")
print(df[df["Marks"] > 80])
```

Output:

```
Student Data:
    Name  Marks
0  Rahul     75
1  Priya     90
2   Aman     65
3   Neha     85
4  Rohit     95

Students Scoring More Than 80 Marks:
    Name  Marks
1  Priya     90
3   Neha     85
4  Rohit     95
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Problem statement 5: Write a program to implement the logistic regression for a sample training data set stored as a .CSV file. Compute the performance evaluation metrics, considering few test data sets.

Explanation: To implement Logistic Regression for classification and evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score, f1_score,
    confusion_matrix )
data = pd.read_csv("Book1.csv")
X = data[["HOURS", "ATTENDENCE"]]
y = data["PASS"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

```
print("Actual Results:", list(y_test))

print("Predicted Results:", list(y_pred))

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Precision:", precision_score(y_test, y_pred, zero_division=0))

print("Recall:", recall_score(y_test, y_pred, zero_division=0))

print("F1 Score:", f1_score(y_test, y_pred, zero_division=0))

print("Confusion Matrix:")

print(confusion_matrix(y_test, y_pred))
```

Output:

```
Actual Results: [0, 1, 1]
Predicted Results: [np.int64(0), np.int64(1), np.int64(1)]
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
Confusion Matrix:
[[1 0]
 [0 2]]
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Problem statement 6: Write a program to demonstrate the working of the decision tree algorithm by considering a dataset.

Explanation: To implement a Decision Tree classifier and predict class labels using input features.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris = load_iris(
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Actual Labels:", y_test)
print("Predicted Labels:", y_pred)
print("Accuracy:", accuracy)
sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(sample)
print("Predicted Flower:", iris.target_names[prediction[0]])
```

Output:

```
Actual Labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Predicted Labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 1.0
Predicted Flower: setosa
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Problem statement 7: Write a Python program to implement Simple Linear Regression for iris using sklearn and plot the confusion matrix.

Explanation: To implement Simple Linear Regression, visualize the regression line, and demonstrate a confusion matrix using a classifier.

Code:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
iris = load_iris()

X = iris.data[:, [2]]
y = iris.data[:, 3]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42 )

reg_model = LinearRegression()
reg_model.fit(X_train, y_train)
y_pred = reg_model.predict(X_test)

print("Actual Petal Width:", y_test[:5])
print("Predicted Petal Width:", y_pred[:5])

plt.scatter(X, y, label="Actual Data")
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

```
plt.plot(X, reg_model.predict(X), color="red", label="Regression Line")
plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("Simple Linear Regression on Iris Dataset")
plt.legend()
plt.show()


X_class = iris.data
y_class = iris.target


X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
   X_class, y_class, test_size=0.2, random_state=42 )
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train_c, y_train_c)


y_pred_c = clf.predict(X_test_c)

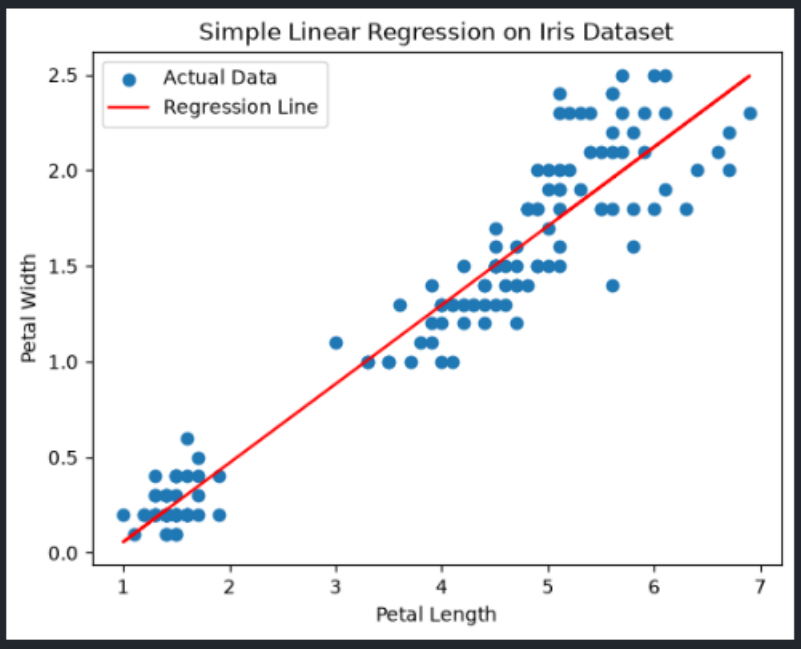

cm = confusion_matrix(y_test_c, y_pred_c)

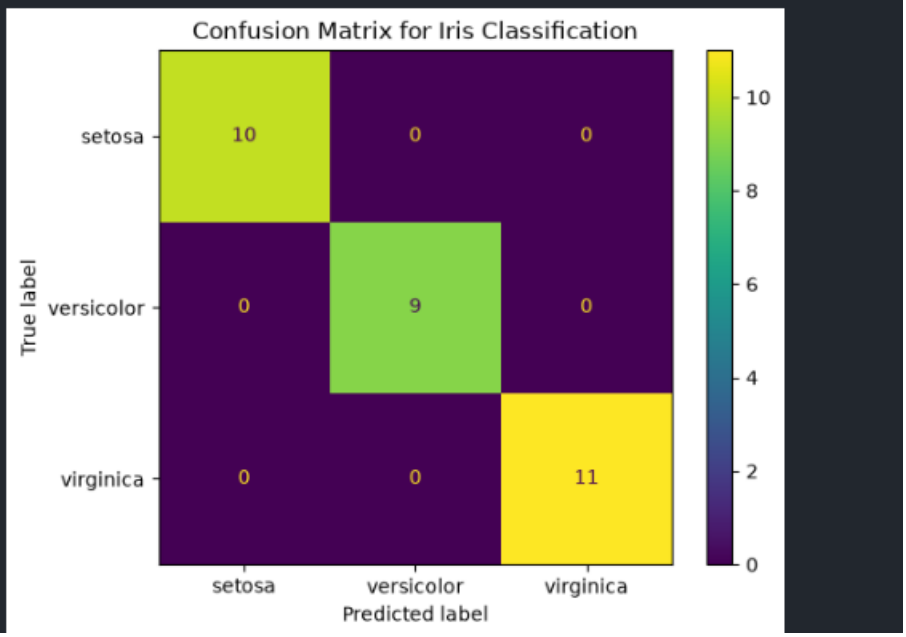

display = ConfusionMatrixDisplay(
   confusion_matrix=cm,
   display_labels=iris.target_names )
display.plot()
plt.title("Confusion Matrix for Iris Classification")
plt.show()
```

Output:

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

```
Actual Petal Width: [1.2 0.3 2.3 1.5 1.4]
Predicted Petal Width: [1.58555194 0.34583706 2.49467619 1.50290428 1.62687577]
```





Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Problem statement 8: Write a python program to implement K-Means clustering Algorithm. Vary the number of k values as follows and compare the results: i. 1 ii. 3 iii. 5

Explanation: To understand unsupervised learning and observe how changing the number of clusters affects the grouping of data.

Code:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
iris = load_iris()
X = iris.data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
for k in [1, 3, 5]:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(X_scaled)
    print("\nNumber of Clusters (K):", k)
    print("Cluster Centers:")
    print(model.cluster_centers_)
    print("Inertia:", model.inertia_)
```

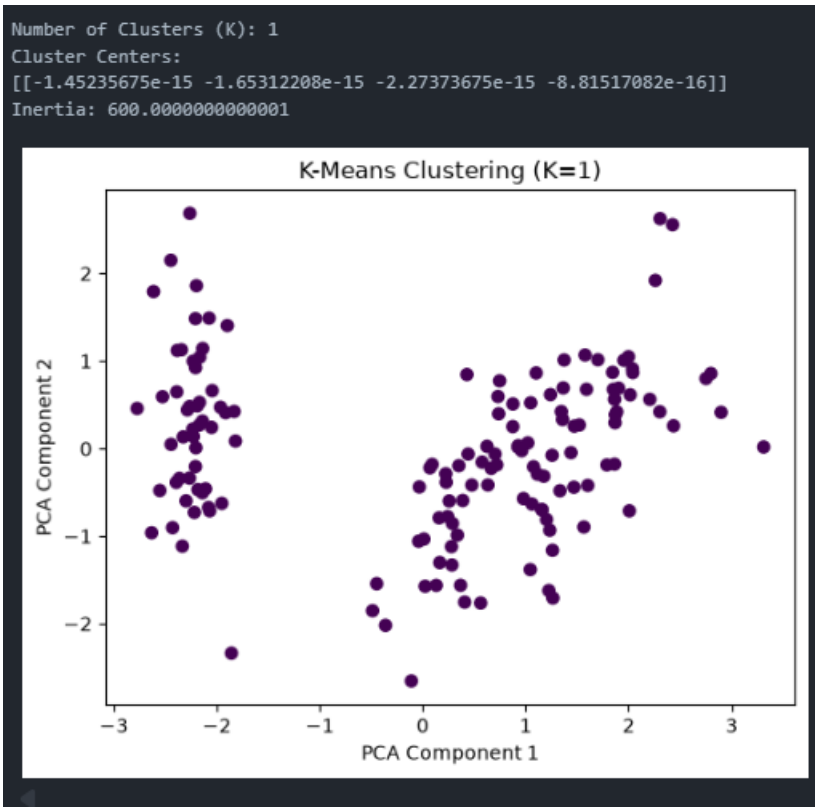
```
    plt.figure()
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

```
    plt.scatter(
        X_pca[:, 0],
        X_pca[:, 1],
        c=labels )
    plt.title(f"K-Means Clustering (K={k})")
    plt.xlabel("PCA Component 1")
    plt.ylabel("PCA Component 2")
    plt.show()
```

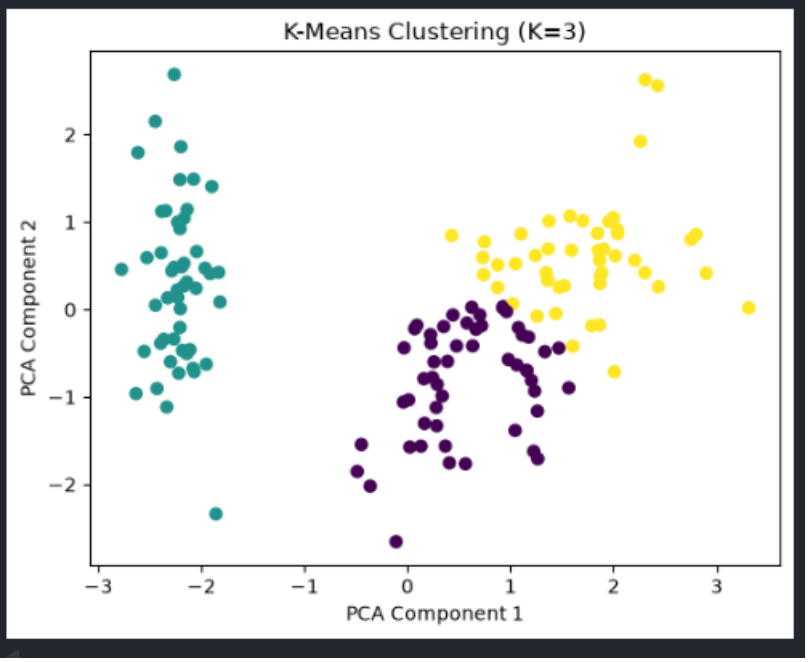
Output:

```
Number of Clusters (K): 1
Cluster Centers:
[[-1.45235675e-15 -1.65312208e-15 -2.27373675e-15 -8.81517082e-16]]
Inertia: 600.0000000000001
```

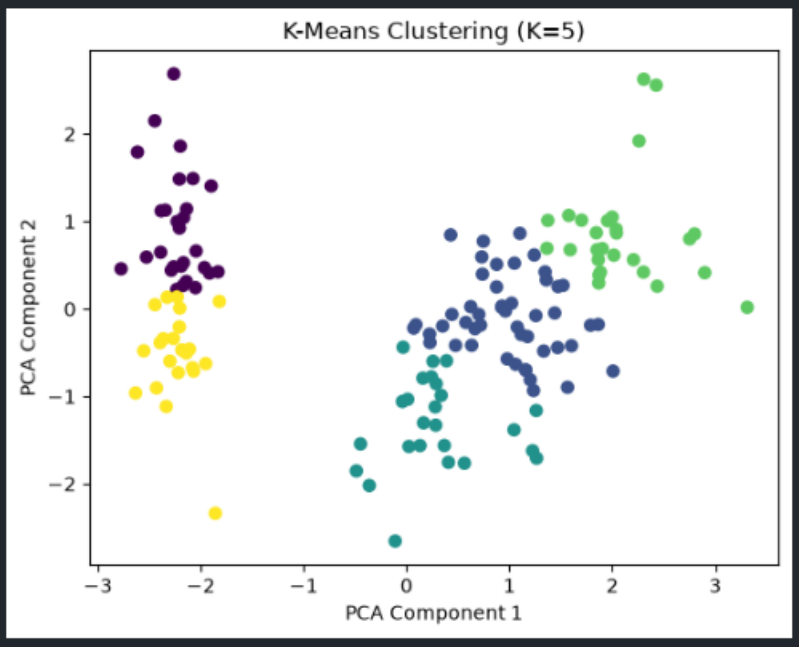


Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

```
Number of Clusters (K): 3
Cluster Centers:
[[-0.05021989 -0.88337647  0.34773781  0.2815273 ]
 [-1.01457897  0.85326268 -1.30498732 -1.25489349]
 [ 1.13597027  0.08842168  0.99615451  1.01752612]]
Inertia: 139.8204963597498
```




```
Number of Clusters (K): 5
Cluster Centers:
[[-0.74922133  1.43007001 -1.29759846 -1.22142131]
 [ 0.44478706 -0.34299318  0.61000768  0.55482966]
 [-0.33846225 -1.30137905  0.13982007  0.05879571]
 [ 1.40150837  0.23463019  1.20272206  1.28307253]
 [-1.35230687  0.11914427 -1.31439133 -1.29749445]]
Inertia: 90.92751382392052
```



Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Problem statement 9: Implement Dimensionality reduction using Principle component Analysis method on a dataset iris.

Explanation: To reduce the number of features in a dataset while preserving as much of its variation as possible and visualize the reduced data.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PC
iris = load_iris()
X = iris.data
y = iris.target
df = pd.DataFrame(X, columns=iris.feature_names)
df["Species"] = [iris.target_names[i] for i in y]


print("Original Dataset:")
print(df.head()
print("\nOriginal Dataset Shape:", X.shape)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
pca_df = pd.DataFrame(
   X_pca,
   columns=["Principal Component 1", "Principal Component 2"] )
pca_df["Species"] = [iris.target_names[i] for i in y]
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

```
print("\nReduced Dataset:")
print(pca_df.head())
print("\nReduced Dataset Shape:", X_pca.shape)
print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)
print(
   "\nTotal Variance Retained:",
   round(sum(pca.explained_variance_ratio_) * 100, 2),
   "%" )
plt.figure(figsize=(9, 6))
for species_id, species_name in enumerate(iris.target_names):
   plt.scatter(
      X_pca[y == species_id, 0],
      X_pca[y == species_id, 1],
      label=species_name, s=60 )
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA: Iris Dataset Reduced to Two Dimensions")
plt.legend()
plt.grid(True)
plt.show()
loadings = pd.DataFrame(
   pca.components_.T,
   index=iris.feature_names,
   columns=["PC1", "PC2"] )
print("\nPCA Component Loadings:")
print(loadings)
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Output:

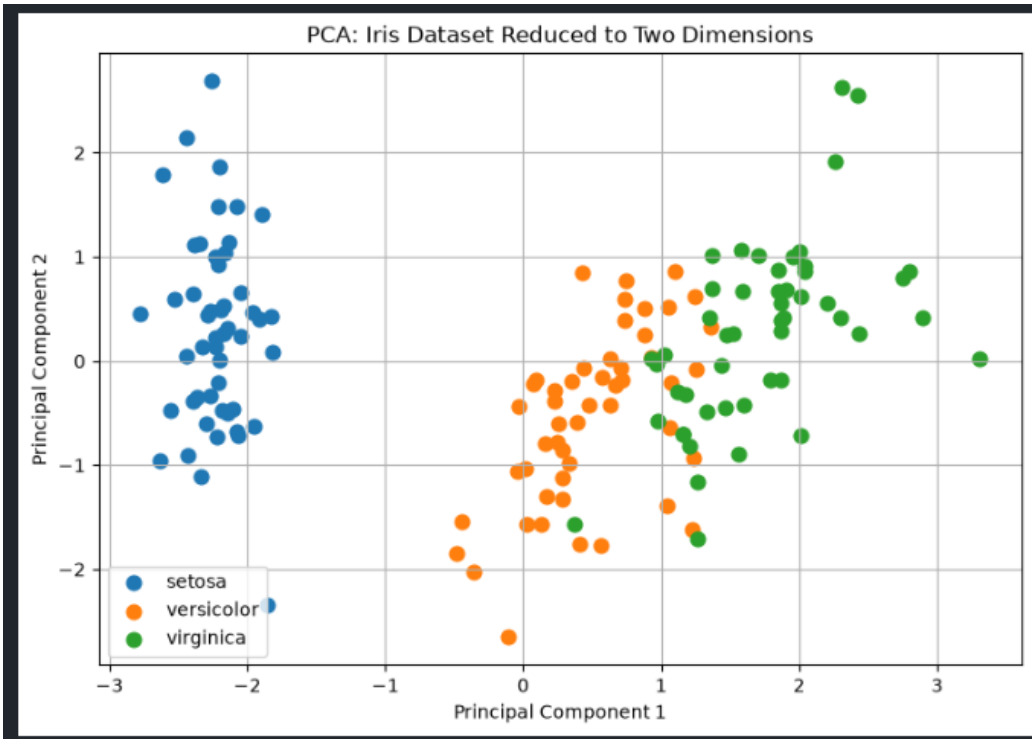
```
Original Dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

  Species
0  setosa
1  setosa
2  setosa
3  setosa
4  setosa

Original Dataset Shape: (150, 4)

Reduced Dataset:
   Principal Component 1  Principal Component 2 Species
0              -2.264703               0.480027  setosa
1              -2.080961              -0.674134  setosa
2              -2.364229              -0.341908  setosa
3              -2.299384              -0.597395  setosa
4              -2.389842               0.646835  setosa
...
Explained Variance Ratio:
[0.72962445 0.22850762]

Total Variance Retained: 95.81 %
```




```
PCA Component Loadings:
                     PC1       PC2
sepal length (cm)  0.521066  0.377418
sepal width (cm)  -0.269347  0.923296
petal length (cm)  0.580413  0.024492
petal width (cm)   0.564857  0.066942
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Problem statement 10: Write a python program to handle missing values in the dataset using Mean, Median, Mode and fillna methods .

Explanation: To understand missing data and learn different techniques for replacing missing values in a Pandas DataFrame.

Code:

```
import pandas as pd
import numpy as np
data = {
   "Name": ["Rahul", "Priya", "Aman", "Neha", "Rohit"],
   "Marks": [80, 90, np.nan, 70, 90],
   "Age": [20, np.nan, 21, 22, 20],
   "City": ["Delhi", "Mumbai", np.nan, "Delhi", "Mumbai"] }

df = pd.DataFrame(data)
print("Original Dataset:")
print(df)

df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
df["Age"] = df["Age"].fillna(df["Age"].median())
df["City"] = df["City"].fillna(df["City"].mode()[0])

print("\nDataset After Handling Missing Values:")
print(df)
print("\nMissing Values in Each Column:")
print(df.isnull().sum())
```

Vishwajeet Rawat
DATE-
BCA (AI & DS)
B2

Output:

```
Original Dataset:
    Name  Marks   Age    City
0  Rahul   80.0  20.0   Delhi
1  Priya   90.0   NaN  Mumbai
2   Aman    NaN  21.0     NaN
3   Neha   70.0  22.0   Delhi
4  Rohit   90.0  20.0  Mumbai

Dataset After Handling Missing Values:
    Name  Marks   Age    City
0  Rahul   80.0  20.0   Delhi
1  Priya   90.0  20.5  Mumbai
2   Aman   82.5  21.0   Delhi
3   Neha   70.0  22.0   Delhi
4  Rohit   90.0  20.0  Mumbai

Missing Values in Each Column:
Name     0
Marks    0
Age      0
City     0
dtype: int64
```